

It's Good to Be Humble: Tamarin Prover's Soundness Versus One Evening with an LLM

Nadim Kobeissi

Symbolic Software

Abstract. The Tamarin prover is among the most theoretically developed tools in symbolic protocol analysis, and it has been used to certify TLS 1.3, 5G-AKA, EMV and Signal. We report five defects in Tamarin 1.12.0 and on the current `develop` branch that cause it to return wrong verdicts, all found in a single evening of LLM-assisted auditing by one person who is not a Tamarin developer.

Three defect families survive scrutiny. First, the predicate that decides whether a restriction is a safety formula accepts formulas containing the `last()` atom, which are not prefix-closed. This makes Tamarin's induction rule unsound: we exhibit one lemma that Tamarin reports *verified* with induction and *falsified* without, and, by chaining through a `[sources]` annotation, a protocol that publishes its key with `Out(k)` whose secrecy Tamarin reports as verified under a clean wellformedness report. The predicate is byte-identical to the 2012 commit that introduced it to fix bug #108, and the `last` atom was already surface syntax in that same commit. The hole and the fix shipped together, and the hole has been open for fourteen years. Second, the wellformedness checks that catch a lemma quantifying over an action fact no rule emits sit behind a guard that inverts their own docstring, and behind a flag hardcoded to `False`; they have not executed since a ProVerif export refactor in PR #725. Third, lemmas annotated `[reuse]` and `[sources]` are admitted as assumptions with no check that they were ever proved, and the assumption constrains both trace quantifiers, so one theory makes Tamarin report *found trace* and *no trace found* for the same statement in one summary block.

We report with equal care what we could not break. Tamarin's constraint solver, its term algebra over exclusive-or, Diffie-Hellman, multisets and natural numbers, its newest monotonicity inference, and its stored-proof replay all resisted every probe we aimed at them. The defects are not in the hard part. Every one sits in the trust boundary around the solver, where a check is skipped rather than an inference drawn wrongly. That is the uncomfortable result: the sophistication is real and the guardrails around it are thin, and an evening was enough to find that out.

Keywords: formal verification · protocol analysis · soundness · Tamarin · tool assurance · LLM-assisted auditing

1 Introduction

Tamarin [1, 2] is a symbolic protocol verifier with unbounded sessions, user-definable equational theories, support for stateful protocols, and a constraint-solving procedure with published soundness and completeness arguments. It is the tool of record for several of the most consequential protocol analyses of the last decade: TLS 1.3 [6], 5G-AKA [7], EMV [8], and others. When a standards body wants a machine-checked argument that a

E-mail: nadim@symbolic.software (Nadim Kobeissi)

protocol meets its goals, Tamarin is one of two or three tools, alongside ProVerif [5], that can supply one.

This paper reports five defects that make Tamarin return wrong answers. All were found in a single evening by one person, working with a general-purpose large language model as an auditing assistant, on a machine with no Tamarin build toolchain installed. We did not read the Tamarin papers in preparation. We did not have prior familiarity with the constraint-solving implementation. What we had was the source tree, a stock binary from Homebrew, and about six hours.

We want to be precise about what that does and does not show, because the temptation to overclaim here is considerable and we have tried to resist it in both directions.

It does not show that Tamarin is a bad tool, or that its theory is wrong, or that published Tamarin results are unreliable. Section 7 reports at length on the parts we attacked and failed to break, and that list is longer than the list of defects. Nor does the `last()` unsoundness invalidate any published case study: we scanned all 1042 models shipped with Tamarin and none of them uses the construct in the position that triggers the bug (§4.6).

What it does show is an asymmetry. Tamarin has been developed for over a decade by a group with deep expertise, and the machinery that group built held up under everything we threw at it. The defects we found are all in the layer around that machinery: whether a formula was classified correctly before being handed to the solver, whether a wellformedness check runs at all, whether a lemma admitted as an assumption was ever proved. Those checks are one-line predicates and list comprehensions. None of them requires a PhD to audit. One of them has been wrong since August 2012.

1.1 Why the cost matters

Finding a soundness bug in a verification tool has historically been expensive. It has usually meant a researcher with the relevant background, motivated by a specific suspicion, spending days or weeks. That cost is why tool assurance has been treated as something that happens occasionally, on purpose, by specialists.

The cost has changed. The audit reported here consisted of reading source with an assistant that could hold the whole call graph in view, forming hypotheses about which predicates were load-bearing, and testing each hypothesis with a ten-line model file. The assistant did not find the bugs by magic and it proposed several theories that turned out to be wrong (§3 describes two). What it changed was the iteration rate. Reading `isSafetyFormula`, noticing that `Last` atoms pass a test named for existentials, and writing the model that exploits it took under twenty minutes.

If one evening at this cost yields five defects in the most theoretically developed tool in the field, the reasonable inference is not that this particular tool is unusually broken. It is that the assumption underneath the whole practice, that tools of this pedigree have been audited to a depth proportional to the trust placed in them, was never tested at a cost anyone could afford. Now it can be.

1.2 Contributions

- Three confirmed defect families in Tamarin 1.12.0 and on `develop` at `ef3f0468`, each reduced to a self-contained model file of at most fifteen lines, each accompanied by a control that isolates its mechanism (§4, §5, §6).
- A fourteen-year provenance analysis of the first defect, establishing by direct comparison of commit contents that the flawed predicate is byte-identical to the one introduced in 2012 to fix the same class of bug, and that the syntax which defeats it was already available in that commit (§4.5).

- A ground-truth-free auditing method (§3.1) in which a tool is caught contradicting itself rather than contradicting a reference answer, which is what makes the third defect checkable by a reader who knows nothing about the protocol under analysis.
- A deliberate account of what resisted attack (§7), including the constraint solver, the term algebra, and stored-proof replay, which is what locates the defects precisely rather than smearing suspicion across the tool.
- An artifact: seven model files, a runner that exits non-zero if any defect stops reproducing, and two analysis scripts (§10).

2 What a Tamarin user is entitled to assume

Tamarin analyzes protocols specified as multiset rewriting rules over a term algebra, in the Dolev-Yao model [4]. A rule consumes facts, emits action facts onto a trace, and produces facts. Security properties are first-order formulas over that trace, written as lemmas. The prover answers `verified` or `falsified` for each lemma, or reports that its analysis is incomplete.

Three features of the input language matter for what follows.

Restrictions constrain the set of traces under consideration. A restriction is a closed formula, and Tamarin analyzes only those traces that satisfy it. Restrictions are how modellers express equality tests, uniqueness of identifiers, and ordering assumptions that the rule language cannot state directly.

Lemma annotations change how a lemma participates in the analysis. A lemma marked `[reuse]` is used as an assumption when proving the lemmas that follow it. A lemma marked `[sources]` is consumed earlier still: it refines the precomputed case distinctions that every other lemma is proved against. A lemma marked `[use_induction]` is proved by induction over the trace rather than by the default simplification.

Wellformedness checks run before analysis and report modelling errors: facts used with inconsistent arity, reserved names, unbound variables. They are the tool's own account of what it thinks is wrong with your model.

A user who writes a model, sees a clean wellformedness report, sees no lemma reported as falsified, sees no `sorry` in the proof, and includes an `exists-trace` lemma to confirm the model is not vacuous has done everything the manual [3] asks. Section 4 exhibits a model that passes every one of those checks and whose secrecy verdict is wrong.

3 Method: one evening, and what the assistant actually did

The audit ran on a MacBook with Tamarin 1.12.0 from Homebrew and Maude 3.5.1. The Tamarin source tree was checked out at `develop` commit `ef3f0468` (cabal version 1.13.0). No Haskell toolchain was installed, so we could not build a modified Tamarin; every hypothesis had to be tested against the stock binary using model files alone. That constraint turned out to be useful, because it forced every finding into a form a user could reproduce.

The working loop was: read a region of source with the assistant, identify predicates whose correctness the rest of the pipeline depends on, and for each one ask what input would make it wrong. Then write a model that supplies that input.

Two things are worth saying plainly about the assistant's role, since the paper's thesis rests on it.

It was not an oracle. Several hypotheses it proposed were wrong, and the wrongness only surfaced under test. It suggested that Tamarin's goal-ranking oracles might prune the search unsoundly; reading `oracleRanking` showed that goals the oracle omits are appended rather than dropped, so the search cannot lose them. It suggested that a heuristic in

observational-equivalence mode might report attacks on unclosed constraint systems; the code supports that reading, but we could not construct a witness, and §7 reports it as an open lead rather than a finding.

What it did supply was breadth at speed. A single reader can hold one hypothesis about one file. Working this way we examined the restriction classifier, the wellformedness gate, the lemma-dependency machinery, the goal manager, the injective-fact inference, the macro and predicate expanders, the proof replay path, and the equational front end, and tested a hypothesis against each. Most produced nothing. The ones that produced something are §§4–6.

3.1 Oracles that need no ground truth

A recurring difficulty in auditing a verifier is that checking its answer requires knowing the answer, which usually means trusting a second tool or a hand proof. We used two invariants instead, both of which catch the tool disagreeing with itself.

The first is quantifier duality. In one theory, state a property twice:

```
lemma all_phi: "phi"
lemma ex_notphi: exists-trace "not phi"
```

Semantics forces these to disagree. If every trace satisfies `phi` then no trace satisfies its negation, and conversely. So `verified/verified` and `falsified/falsified` are both self-contradictions, detectable without knowing which answer is right. This invariant is what makes the defect in §6 checkable by a reader who has never seen the protocol.

The second is annotation neutrality. Options documented to affect only how the search runs, and attributes documented to be sound, must not change a verdict. Deleting a `[reuse]` token, or rephrasing a restriction into a logically equivalent form, must leave every verdict where it was. Each control in §§4–6 is an instance of this.

We built harnesses for two further invariants that did not pay off within the evening: agreement across the sixteen documented configuration flags, and symmetry of observational equivalence under swapping the two sides of every `diff` term. Both sweeps exhausted available memory on real case studies and were killed before completing. We ship them (§10) and claim nothing on their behalf.

4 Defect 1: `last()` defeats the safety-formula classifier

4.1 The mechanism

Tamarin's induction rule is sound only over a prefix-closed set of traces. The implementation knows this. In `formulaToSystem` (`lib/theory/src/Theory/Constraint/System.hs:849-856`), restrictions are split in two, with the reason attached as a comment:

```
(safetyRestrictions, otherRestrictions) =
  partition isSafetyFormula restrictions
-- Non-safety restrictions must be added to the formula, as they
-- render the set of traces non-prefix-closed, which makes the use
-- of induction unsound.
gf2 = gconj $ gf1 : otherRestrictions
```

Safety restrictions go into `sLemmas` and are asserted alongside the proof obligation. Non-safety restrictions are conjoined into the obligation itself, before induction is applied. The split is correct, and it is the fix for a soundness bug reported in 2012 (§4.5).

Everything therefore depends on the classifier, defined in `Guarded.hs` at lines 156 to 164:

```
isSafetyFormula gf0 = null (frees [gf0]) && noExistential gf0
  where
    noExistential (GAto _) = True
```

```

theory LastInductionUnsound begin

rule Start: [ Fr(x) ] --[ Start(x) ]-> [ A(x) ]
rule Step: [ A(x) ] --[ Step(x) ]-> [      ]

restriction StartIsNotLast:
  "All x #i. Start(x) @ i ==> not(last(#i))"

lemma NoStart_with_induction [use_induction]:
  "All x #i. Start(x) @ i ==> F"

lemma NoStart_without_induction:
  "All x #i. Start(x) @ i ==> F"

lemma StartReachable: exists-trace
  "Ex x #i. Start(x) @ i"

end

```

Figure 1: The minimal witness. The two NoStart lemmas are identical formulas.

```

noExistential (GGuarded Ex _ _ _) = False
noExistential (GGuarded All _ _ gf) = noExistential gf
noExistential (GDisj disj) = all noExistential $ getDisj disj
noExistential (GConj conj) = all noExistential $ getConj conj

```

The test is: closed, and free of existential quantifiers. The docstring above it states the property it is standing in for, that a trace violating a safety formula can never be extended into one satisfying it.

Closed and existential-free is not the same property. Tamarin’s formula syntax includes the atom `last(#i)`, which holds when `#i` is the maximal timepoint of the trace. It is documented user-facing syntax ([manual/src/011_advanced-features.md:576](#)) and is parsed at `Theory/Text/Parser/Formula.hs:46`. It contains no existential quantifier. So the restriction

```

restriction StartIsNotLast:
  "All x #i. Start(x) @ i ==> not(last(#i))"

```

passes `isSafetyFormula`, and Tamarin annotates it `// safety formula` in its own output. It is not one. The trace `[Start(x)]` violates it, because `Start` is the final event. Its extension `[Start(x), Step(x)]` satisfies it. A violating trace extends into a satisfying one, which is the negation of the property the docstring names.

4.2 The consequence

Figure 1 is the whole model. Both lemmas are the same formula.

Tamarin’s verdict:

```

NoStart_with_induction    (all-traces): verified (3 steps)
NoStart_without_induction (all-traces): falsified - found trace (2 steps)
StartReachable            (exists-trace): verified (2 steps)
/* All wellformedness checks were successful. */

```

The property is false: `[Start(x), Step(x)]` satisfies the restriction and contains a `Start`. Tamarin agrees when it does not use induction. The `exists-trace` lemma confirms the model is not vacuous.

4.3 The control

A reader might reasonably suspect the protocol, the lemma, or induction in general. The control rules all three out. Take the same protocol and the same lemma, and phrase the restriction as a guarded existential:

```

theory LastSourcesChain begin

rule Gen: [ Fr(k) ] --[ Gen(k) ]-> [ Out(k) ]
rule Recv: [ In(x) ] --[ Recv(x) ]-> [          ]

restriction GenIsNotLast:
  "All k #i. Gen(k) @ i ==> not(last(#i))"

lemma bogus_sources [sources]:
  "All k #i. Gen(k) @ i ==> F"

lemma secrecy:
  "All k #i #j. Gen(k) @ i & KU(k) @ j ==> F"

lemma gen_reachable: exists-trace
  "Ex k #i. Gen(k) @ i"

end

```

Figure 2: The key is published by `Out(k)`. Tamarin reports its secrecy as verified.

```

restriction StartImpliesStep:
  "All x #i. Start(x) @ i ==> Ex #j. Step(x) @ j"

```

This restriction is also not prefix-closed: $[\text{Start}(x)]$ violates it and $[\text{Start}(x), \text{Step}(x)]$ satisfies it, exactly as before. The difference is that it contains an existential, so `isSafetyFormula` returns `False` and Tamarin conjoins it into the obligation. The verdict:

```

NoStart_with_induction (all-traces): falsified - found trace (6 steps)
NoStart_without_induction (all-traces): falsified - found trace (4 steps)

```

and the restriction carries no `// safety formula` annotation. Tamarin handles non-prefix-closed restrictions correctly when it recognises them. The defect is the recognition, and nothing else.

We note in passing that the most literal rendering of $\neg \text{last}(i)$, namely `Ex #j. #i < #j`, is rejected by Tamarin as an ungarded existential, which is why the control uses an action atom as the guard.

4.4 From a toy to a wrong security verdict

Figure 1 needs an explicit `[use_induction]`, which a reviewer might notice. The annotation is not necessary. Two facts combine.

First, `[sources]` implies induction with no annotation from the user (`lib/theory/src/ClosedTheory.hs:120`):

```

inductionHint
| any (`elem` [SourceLemma, InvariantLemma])
  (L.get lAttributes l) = UseInduction
| otherwise
  = AvoidInduction

```

Second, `[sources]` lemmas are consumed during theory closing, before any proof runs, and the case distinctions they refine are then used by every lemma in the file (`CloseRule.hs:426`).

So a single misclassified restriction can wrongly prove a false sources lemma, which then prunes legitimate cases out of the whole theory. Figure 2 is the result.

```

bogus_sources (all-traces): verified (3 steps)
secrecy (all-traces): verified (2 steps)
gen_reachable (exists-trace): verified (2 steps)
/* All wellformedness checks were successful. */

```

Rule `Gen` sends the fresh key on the public channel. The adversary learns it immediately. Tamarin closes `secrecy` with `by solve( !KU( n ) @ #j )`: the false refinement removed the

only case that mattered.

Every signal the manual teaches a user to check is green. The wellformedness report is clean. No proof contains a `sorry`. No lemma is reported falsified. The `exists-trace` lemma confirms the model is live, so the usual vacuity defence does not apply. Deleting only the sources lemma restores `secrecy: falsified`, which confirms the mechanism.

4.5 Provenance: the fix and the hole shipped together

In August 2012 Simon Meier committed a model file titled `Axioms_and_Induction.spthy` with the message “example illustrating soundness bug #108; i.e., that our current handling of axioms and trace induction is not sound” (`a32cc3f8`). The model is still in the tree at `examples/loops/Axioms_and_Induction.spthy`, and its comment reads “Note that the set of traces satisfying this trace is **not** prefix-closed.” Six days later, commit `9726e4c4`, “fixed two soundness bugs”, introduced `isSafetyFormula` and the partition that uses it.

We compared the function body at that commit against the current one:

```
git show 9726e4c4:src/Theory/Constraint/System/Guarded.hs
git show HEAD:lib/theory/src/Theory/Constraint/System/Guarded.hs
```

The two are byte-identical. The predicate has never been modified in fourteen years.

The question this raises is whether `last` was reachable from surface syntax in 2012, or whether the hole opened later when the atom was exposed. It was reachable. The same commit that introduced the fix contains, at `src/Theory/Text/Parser.hs:340`:

```
[ Last <$> try (symbol "last" *> parens nodevarTerm) <?> "last atom"
```

The fix for bug #108 and the syntax that defeats it were in the same commit, on the same day. The 2012 counterexample used an existential, which the new predicate caught. The `last` phrasing of the same idea was already expressible and was not caught, and has not been caught since.

4.6 Scope, stated against our own interest

The defect is real and it is reachable from documented syntax. It does not invalidate any published Tamarin result, and we think a paper that implied otherwise would deserve to be dismissed.

We scanned every model shipped with Tamarin: 1042 files under `examples/`. Thirty-six contain the string `last(`. In every one, the occurrence is inside Tamarin’s own machine-generated proof text, saved into an analyzed output, of the form

```
solve( (last(#i)) | (Ex #j. Update(k,r) @ #j & !(last(#j)) & #j < #i) )
```

None is a user-written restriction. Zero models use `last()` in the position that triggers the bug. The scanner is `attacks/tools/lastscan.py`.

So the accurate claim is that this is a live unsoundness in documented syntax that currently affects no published result. It is a trap that has been armed for fourteen years and that nobody has stepped on, which is a different and less comfortable statement than either “Tamarin’s results are wrong” or “this does not matter”.

4.7 The fix

One line, in `Guarded.hs`:

```
noExistential (GAto (Last _)) = False
```

`toInductionHypothesis` already refuses formulas containing `last` (`Guarded.hs:619`, error string “formula not last-free”), so reclassifying makes induction inapplicable to theories carrying such a restriction, which is the correct behaviour.

```

theory VacuousLemmaTypo begin

rule Setup: [ Fr(k) ] --[ SecretKey(k) ]-> [ Key(k) ]
rule Leak: [ Key(k) ] --[ Leaked(k) ]-> [ Out(k) ]

// The rules emit SecretKey. This lemma says SecretKey.
lemma secrecy_typo:
  "All k #i. SecretKey(k) @ i ==> not(Ex #j. K(k) @ j)"

lemma secrecy_intended:
  "All k #i. SecretKey(k) @ i ==> not(Ex #j. K(k) @ j)"

end

```

Figure 3: A one-character typo. Tamarin reports the vacuous lemma as verified and the wellformedness report as clean.

5 Defect 2: the vacuity checks have not run since PR #725

The most common modelling error in Tamarin is a lemma that quantifies over an action fact no rule emits. Such a lemma is vacuously true and Tamarin reports it verified. Practitioners are taught to guard against this by hand, but Tamarin also ships two checks for it, `inexistentActions` and `inexistentActionsRestrictions` (`Theory/Tools/Wellformedness.hs:706-734`), fully implemented and with error messages written.

They do not run. Here is the call site (`Wellformedness.hs:578-583`):

```

-- | Report on facts usage. Skip checks on non-existent actions
-- if `incompleteMSRs` is True.
factReports incompleteMSRs thy =
  concat [ reservedReport, ..., factUsage, factLhsOccurNoRhs ]
  ++ concat [ inexistentActions ++ inexistentActionsRestrictions
             | incompleteMSRs ]

```

The list comprehension *includes* the checks when `incompleteMSRs` is `True`, which is the inverse of what its own docstring says. And the sole production call site fixes the flag (`src/Main/TheoryLoader.hs:602`):

```

incompleteMSRs = False
-- TODO how do we know if we do not have all MSRs due to translation?

```

Either reading condemns the code. Under the docstring's intent the guard is backwards; under the guard's literal meaning the flag is set wrongly. In both cases the checks are unreachable in the shipped tool.

5.1 Witness

```

secrecy_typo      (all-traces): verified (2 steps)
secrecy_intended  (all-traces): falsified - found trace (3 steps)
/* All wellformedness checks were successful. */

```

Running with `--quit-on-warning` changes nothing, because no warning is produced: the check that would produce it never executes.

5.2 Provenance

`git log -L 583,583` on the file attributes the guard to commit `98c3887f`, “Extend ProVerif export module (#725)”. We diffed the commit directly. Before it:

```

factReports :: OpenTranslatedTheory -> WfErrorReport
factReports thy = concat

```



```
[ reservedReport, reservedFactNameRules, freshFactArguments
, specialFactsUsage, factUsage, factLhsOccurNoRhs
, inexistentActions, inexistentActionsRestrictions ]
```

After it, the last two members moved behind the guard shown above. A refactor of an unrelated export backend silently disabled the tool's only automatic guard against its most common modelling error, and no test caught the regression.

6 Defect 3: unproven assumptions are admitted as facts

6.1 The mechanism

`gatherReusableLemmas` (`lib/theory/src/CloseRule.hs:181-188`) decides which lemmas are admitted as assumptions:

```
gatherReusableLemmas kind = do
  LemmaItem lem <- previousItems
  guard $ lemmaSourceKind lem ≤ kind
    && ReuseLemma `elem` L.get lAttributes lem
    && AllTraces = L.get lTraceQuantifier lem
    && L.get lName lem `notElem` L.get pcHiddenLemmas ctxt
    && "ALL" `notElem` L.get pcHiddenLemmas ctxt
  return $ formulaToGuarded_ $ L.get lFormula lem
```

The filter checks the attribute, the trace quantifier, the source kind and the hide-list. It does not check whether the lemma was proved. We looked for a proof-status gate elsewhere on the path and there is none: the `CompleteProof` constructor is consumed only in `src/Web/Theory.hs`, where it colours a cell in the interactive interface.

The comment beside the insertion states the justification:

```
-- Note that it is OK to add reusable lemmas directly to the system,
-- as they do not change the considered set of traces. This is the key
-- difference between lemmas and restrictions.
```

That is sound under the precondition that the lemma holds. Nothing enforces the precondition.

6.2 Tamarin contradicting itself

The admitted formulas go into `sLemmas`, which constrains the constraint system for both trace quantifiers. A false assumption therefore does two things at once: it closes all-traces branches that should stay open, and it prunes the witness search for exists-trace lemmas. Figure 4 exploits both.

```
helper (all-traces): falsified - found trace (3 steps)
target (all-traces): verified (2 steps)
live (exists-trace): falsified - no trace found (2 steps)
```

Read the first and third lines together. `helper` is `All k #i #j. Secret(k)@i & KU(k)@j => F` and `live` is `Ex k #i #j. Secret(k)@i & KU(k)@j`. The second is the negation of the first. “Falsified” on `helper` asserts that such a trace exists. “Falsified” on `live` asserts that no such trace exists. Tamarin affirms and denies the same proposition in one summary block.

This is the defect we consider most serious, and not because its consequences are the worst. It is the one that requires nothing of the reader. No knowledge of the protocol, no judgement about what the right answer is, and no trust in us. Two lines of the tool's own output are inconsistent with each other.

```

theory ReusePoisonsBothDirections begin

rule Gen: [ Fr(k) ] --[ Secret(k) ]-> [ Out(k) ]

lemma helper [reuse]:
  "All k #i #j. Secret(k) @ i & KU(k) @ j ==> F"

lemma target:
  "All k #i. Secret(k) @ i ==> not(Ex #j. KU(k) @ j)"

lemma live: exists-trace
  "Ex k #i #j. Secret(k) @ i & KU(k) @ j"

end

```

Figure 4: helper and live are exact negations of one another.

Why is the inconsistency visible at all, given that both poisoned verdicts might have been consistent with one another? Because `previousItems` draws only from lemmas that precede the current one, so `helper` never reuses itself and receives the correct verdict, while every later lemma is evaluated under the refuted assumption. The pair `target/live`, by contrast, comes back `verified/falsified`, which is mutually consistent and therefore invisible to the duality oracle of §3.1. Detecting this class requires pairing against the `reuse` lemma itself.

6.3 The control

Deleting the single token `[reuse]` from Figure 4, changing nothing else, gives

```

helper (all-traces):  falsified - found trace (3 steps)
target (all-traces):  falsified - found trace (3 steps)
live  (exists-trace): verified (3 steps)

```

all three correct. This fixes the ground truth, so the contradiction above resolves as follows: `live` is true, `target` is false, and Tamarin's verdicts for both are wrong.

6.4 The same defect on a hotter path

`[sources]` lemmas are admitted the same way and consumed earlier, during theory closing, with no failure path if their proof later fails. The blast radius is the whole theory rather than one lemma:

```

bogus_sources (all-traces): falsified - found trace (2 steps)
target        (all-traces): verified (2 steps)

```

The auto-generated sources lemmas produced by `--auto-sources` take the same unguarded path. We checked whether this is realised in practice: all sixteen auto-sources case studies in `case-studies-regression/` prove their generated `AUTO_typing` lemma, so the risk there is structural rather than current.

6.5 On invalidation

Tamarin has a proof status for exactly this hazard. `Theory/Proof.hs:406` defines `InvalidatedProof`, documented as “The proof has been Invalidated (eg. by editing a reuse lemma)”. The constructor is set in exactly one place in the codebase: `src/Web/Handler.hs:255`, the interactive web interface.

We want to state the consequence carefully, because the obvious framing is wrong. The manual documents this invalidation in its interactive-mode section (`manual/src/007_property-specification.md:229-234`), so batch mode never promised it.

The correct statement is not that a safety net fails to fire. It is that the safety net exists only in the graphical interface, while batch mode, which is what continuous integration and every published artifact use, has no equivalent and no proof-status gate anywhere on the reuse path.

7 What we could not break

This section is longer than any of the preceding three, which is the point.

We report it for two reasons. Negative results locate a defect: knowing that the solver held while the classifier failed is what turns five bug reports into a claim about where assurance effort is misallocated. And a paper that reported only hits would deserve the suspicion that it went looking for a conclusion.

The constraint solver and term algebra. The duality oracle of §3.1 was run over fifteen models covering exclusive-or (self-cancellation, one-time pad, chained pads, the zero constant), multisets, Diffie-Hellman, subterms, natural numbers, hashing, and symmetric encryption. No contradictions. Controls in the same battery return opposite verdicts as they should, so the oracle discriminates.

Injective facts and stateful reasoning. `simpInjectiveFactEqMon` (`Simplify.hs:534-660`) is the newest and most aggressive inference in the solver: it derives equalities and ordering constraints from inferred monotonic behaviour of fact arguments, which is the dangerous direction, since it adds constraints rather than removing them. It held on constant positions, strictly increasing positions, natural-number counters, non-fresh identifiers, and SAPIC locked and unlocked state cells.

Stored-proof replay. Tamarin serialises proofs into the model file, and we expected replay to be a soft target. It is not. Three separate forgeries against `case-studies-regression/Tutorial_analyzed.spthy`, namely weakening a lemma statement while keeping its proof, replacing a proof with a bogus one-step by contradiction, and relabelling a case, were all caught and downgraded to `analysis incomplete`. Published proof artifacts do get re-validated on load.

Goal-ranking oracles. `oracleRanking` (`ProofMethod.hs:604-621`) returns `ranked ++ remaining`. Goals the external oracle omits are appended rather than dropped, so a hostile or buggy oracle costs time and not soundness. The only termination path, `quitOnEmpty`, emits a `Sorry` and is reported as an incomplete proof.

Predicate expansion. `expandFormula` (`Theory/Syntactic/Predicate.hs:94-105`) shifts de Bruijn indices correctly. A predicate body binding `y`, used at a call site that also binds `y`, gives results identical to a non-colliding control. Macro expansion, by contrast, is not hygienic: a macro body variable is captured by a same-named rule variable, silently, with a clean wellformedness report. We do not report that as a defect, because the genuinely dangerous case, a macro injecting an unbound variable, is caught by `unboundReport`, and because we could not turn the capture into a wrong verdict.

Goal management. The `openGoals` filter drops `KU` goals it treats as always constructible (`Goals.hs:70-84`). We checked whether the dropped goals are genuinely handled elsewhere; they are, decomposed into sub-goals by `insertAction` (`Reduction.hs:293-350`).

Equational front end. Non-terminating equations are rejected at parse time. Equations that are not subterm-convergent produce an explicit warning naming the offending equation. Precomputation limits (`-s`, `-c`) leave sources under-refined when hit, which costs completeness and not soundness.

An open lead. In observational-equivalence mode, `DiffAttack` (`ProofMethod.hs:373-387`) can report an attack on a constraint system it never closed:

```
guard (solved || (trivial sys' && notContradictory))
```

The load-bearing assumption is that a goal over pairwise-distinct message variables is always satisfiable. It is asserted in a comment, and it has failed before: commit `37c7285a` is titled “fixed bug when mirroring XOR leading to false attacks”, and `07021243` narrowed the same check with the message “otherwise unsound”. We could not construct a witness. Our two attempts, a reflexivity check on `diff(t,t)` and a symmetry sweep over real case studies, produced one completed passing case and one sweep that exhausted memory before finishing. This lead is open, not cleared. It is also the most promising direction for the one result this paper lacks, a false attack rather than a missed one.

7.1 The shape of the result set

Every confirmed defect makes Tamarin too permissive. Not one makes it too strict. For a tool whose output is used to certify deployed protocols this is the worse direction, and it is consistent with the pattern: each defect is a place where a check was skipped, never a place where an inference was drawn incorrectly. The inferences are, as far as we could determine in an evening, correct.

8 Disclosure

We attempted coordinated disclosure and could not find a channel to use.

At the time of writing, the Tamarin repository has no security policy. There is no `SECURITY.md` at the repository root or under `.github/`, so GitHub’s advisory interface at `github.com/tamarin-prover/tamarin-prover/security` offers no reporting path. `CONTRIBUTING.md` documents pull requests and issue etiquette and does not mention vulnerabilities, security reports, or private disclosure. The `.github/` directory contains two workflow files and nothing else.

For most projects the absence of a policy is unremarkable. For a tool whose output is cited in support of the security of deployed protocols, a defect that turns `falsified` into `verified` is closer to a vulnerability than to an ordinary bug, and the absence of a route to report one privately is worth naming.

We are therefore publishing directly, with complete reproductions, and we would encourage the maintainers to adopt a policy. Our first recommendation in §9 is that one be added, because it costs a file and it is the precondition for everything else in that list.

9 What we think this means

9.1 Sophistication and assurance are not the same axis

Tamarin’s constraint-solving procedure is a serious piece of work with a published metatheory, and our evening produced no evidence against it. The defects are all in code that any competent programmer could have audited: a five-line predicate, an inverted list comprehension, a filter missing a conjunct. This is not a coincidence, and we do not think it is specific to Tamarin.

Attention follows interest. The constraint solver is where the research is, so it gets the papers, the proofs, and the scrutiny. The restriction classifier is plumbing. But the classifier is what decides whether the solver’s soundness theorem applies to your model, and a soundness theorem about a set of traces you are not analyzing is not a soundness theorem about your protocol. The guardrails carry the theorem across to the user, and they had not been audited to anything like the depth of the thing they carry.

9.2 A regression test would have caught two of three

The 2012 counterexample for bug #108 is still in the tree. Nobody added its `last-phrased` sibling. The vacuity checks were unconditional before PR #725 and gated afterwards, and no test noticed, because the test suite compares end-to-end verdicts on case studies and no case study depends on those checks firing.

Both are cheap to prevent. A property test asserting that `isSafetyFormula` implies prefix-closure on generated formulas would have caught the first at any point in fourteen years. A single model with a deliberately misspelled action fact, asserting that the wellformedness report is non-empty, would have caught the second at the moment of the refactor.

9.3 On the cost of an audit

The claim in our title is meant literally, and we want to be careful with it in both directions.

An evening was enough. It was enough because reading unfamiliar Haskell with an assistant that can follow a call graph across twenty files is much faster than reading it alone, and because the hypotheses that mattered were cheap to test: a ten-line model file and a sub-second run. The assistant was wrong repeatedly, and every finding here survived because it was reduced to a reproduction rather than to an argument. That is the part of the method we would defend most strongly, and it is not new; it is just newly cheap.

An evening was also not enough for plenty. We did not complete the configuration-agreement sweep. We did not complete the observational-equivalence symmetry sweep. We did not seriously examine the SAPIC translation, the export backends, or the accountability machinery. We have one open lead and no false attack. A determined effort over a week, or the same method applied by people who know the codebase, would find more.

The lesson we would draw is not about Tamarin, and it is certainly not that tools like it should be trusted less in favour of nothing. It is that the expected cost of auditing them has dropped by something like an order of magnitude, and the field has not yet adjusted its habits to that fact. Every verifier in common use, ours included, deserves an evening of this. We expect most of them will yield something, and we expect that to be good for all of us.

9.4 Recommendations

1. Add a `SECURITY.md` with a private reporting address.
2. Fix `isSafetyFormula` by rejecting `last` atoms, one line, and add the `last-phrased` sibling of the 2012 counterexample to the regression suite.
3. Repair the `incompleteMSRs` guard so the vacuity checks execute, and add a model whose wellformedness report is asserted non-empty.
4. Refuse to admit a `[reuse]` or `[sources]` lemma whose proof is not closed, or propagate a taint marker into dependent lemmas so the summary reads `verified (assuming: helper)` rather than `verified`.

5. Add property tests for the predicates the soundness argument depends on, separately from the end-to-end case-study suite, since the latter cannot see a check that stopped running.

10 Artifact

The artifact is seven Tamarin model files, a runner, and two analysis scripts. Five models reproduce defects and two are controls. `run.sh` executes all seven, checks each verdict against the expected pattern, and exits non-zero if any defect stops reproducing, so it doubles as a regression test against a patched build. Every model completes in under a second.

`tools/lastscan.py` produces the corpus scan of §4.6. `tools/consistency_oracle.py` implements the duality oracle of §3.1 and contains the fifteen-case battery of §7. We also ship the two harnesses whose sweeps did not complete (`differ.py`, `diffsym.py`); they are included because the invariants they encode are the right ones, and we claim no result on their behalf.

All findings were verified against `tamarin-prover 1.12.0` with Maude 3.5.1, and every implicated code path was compared across `1.12.0..HEAD` and is unchanged, so the defects are present on `develop` at `ef3f0468`.

11 Related work

Soundness defects in verification tools have been reported before, and the genre is well established: implementations of decision procedures have been found to disagree with their own metatheory in SMT solvers [10, 11] and in C compilers [9], generally by differential or metamorphic testing rather than by proof. Our duality oracle (§3.1) is a metamorphic relation in that tradition, specialised to a tool that answers questions about trace sets.

Within protocol verification, comparisons between tools have surfaced discrepancies that turned out to be tool defects rather than modelling differences, and Tamarin's own history records several: the two soundness bugs fixed in `9726e4c4`, the mirroring bug in `37c7285a`, and the narrowing in `07021243`, each found by the developers themselves. What we add is not a new class of defect but a data point about cost: the same kind of finding, obtained without domain expertise in the tool, in one evening.

12 Conclusion

We found five defects in Tamarin in one evening. Three are serious: an unsound induction rule reachable from documented syntax and open since 2012, a pair of wellformedness checks that have not executed since an unrelated refactor, and a lemma-dependency mechanism that admits refuted assumptions and can be made to affirm and deny the same proposition in a single run.

We also spent that evening failing to break the constraint solver, the term algebra, the monotonicity inference, the goal manager, and proof replay. That failure is the more interesting half of the result. The hard parts of Tamarin are in good shape. The checks that decide whether those hard parts apply to your model are not, and nobody had looked, because looking used to be expensive and now it is not.

None of this makes Tamarin a bad tool. We would still reach for it. It does suggest that the confidence the field places in tools of this class has been calibrated against their theory rather than against their implementations, and that the gap between the two is now cheap enough to measure that there is no longer an excuse for leaving it unmeasured.

That applies to every tool in this space, including our own. It seemed worth an evening to find out. We would recommend the exercise.

References

- [1] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification (CAV)*, LNCS 8044, pp. 696–701, 2013. https://doi.org/10.1007/978-3-642-39799-8_48.
- [2] B. Schmidt, S. Meier, C. Cremers, and D. Basin. Automated Analysis of Diffie-Hellman Protocols and Advanced Security Properties. In *IEEE Computer Security Foundations Symposium (CSF)*, pp. 78–94, 2012. <https://doi.org/10.1109/CSF.2012.25>.
- [3] The Tamarin Team. *Tamarin-Prover Manual: Security Protocol Analysis in the Symbolic Model*. <https://tamarin-prover.com/manual/>. Accessed August 2026.
- [4] D. Dolev and A. C. Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983. <https://doi.org/10.1109/TIT.1983.1056650>.
- [5] B. Blanchet. An Efficient Cryptographic Protocol Verifier Based on Prolog Rules. In *IEEE Computer Security Foundations Workshop (CSFW)*, pp. 82–96, 2001. <https://doi.org/10.1109/CSFW.2001.930138>.
- [6] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM Conference on Computer and Communications Security (CCS)*, pp. 1773–1788, 2017. <https://doi.org/10.1145/3133956.3134063>.
- [7] D. Basin, J. Dreier, L. Hirschi, S. Radomirovic, R. Sasse, and V. Stettler. A Formal Analysis of 5G Authentication. In *ACM Conference on Computer and Communications Security (CCS)*, pp. 1383–1396, 2018. <https://doi.org/10.1145/3243734.3243846>.
- [8] D. Basin, R. Sasse, and J. Toro-Pozo. The EMV Standard: Break, Fix, Verify. In *IEEE Symposium on Security and Privacy (S&P)*, pp. 1766–1781, 2021. <https://doi.org/10.1109/SP40001.2021.00037>.
- [9] X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and Understanding Bugs in C Compilers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 283–294, 2011. <https://doi.org/10.1145/1993498.1993532>.
- [10] D. Winterer, C. Zhang, and Z. Su. On the Unusual Effectiveness of Type-Aware Operator Mutations for Testing SMT Solvers. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA), Article 193, pp. 193:1–193:25, 2020. <https://doi.org/10.1145/3428261>.
- [11] R. Brummayer and A. Biere. Fuzzing and Delta-Debugging SMT Solvers. In *7th International Workshop on Satisfiability Modulo Theories (SMT)*, pp. 1–5, 2009. <https://doi.org/10.1145/1670412.1670413>.